

API Gateway – Week 2 Design Documentation

This document captures the **design decisions, rationale, and learning references** for **Week 2 – Day 1 and Day 2** of the Job Portal microservices project.

The goal of this phase is to build a **production-grade API Gateway foundation** before introducing business logic.

1. Context & Big Picture

Role of the API Gateway

The API Gateway acts as the **single entry point** to the backend system. Its responsibilities are intentionally limited:

- Receive client (frontend) requests
- Route requests to appropriate backend services
- Handle failures and timeouts consistently
- Remain thin, stateless, and scalable

What the gateway must NOT do: - Contain business logic - Directly access databases - Know implementation details of downstream services

This design follows industry-standard **microservices gateway patterns**.

2. Day 1 – Gateway Skeleton & Configuration

2.1 Folder Structure

The gateway uses a layered structure:

- `main.py` → Application entry point
- `core/` → Configuration and cross-cutting concerns
- `routes/` → HTTP endpoints (no logic)
- `services/` → Communication with downstream services

Why this is best: - Enforces separation of concerns - Keeps `main.py` thin - Scales cleanly as the application grows

This structure is directly inspired by **FastAPI's official "Bigger Applications" recommendation**.

2.2 Centralized Configuration (`config.py`)

Service URLs are loaded via environment variables:

- `AUTH_SERVICE_URL`
- `USER_SERVICE_URL`
- `JOB_SERVICE_URL`
- `APPLICATION_SERVICE_URL`

Why this is best practice: - Avoids hardcoded service locations - Supports multiple environments (local, Docker, cloud) - Follows the **12-Factor App – Config principle**

The gateway is the **only component aware of service locations**, keeping other services isolated.

2.3 Docker Compose Environment Injection

Environment variables are defined **only for the API Gateway** in `docker-compose.yml`.

Why only the gateway: - Prevents tight coupling between services - Ensures services remain independently deployable - Matches real-world microservices architecture

3. Day 2 – HTTP Client Abstraction

3.1 Problem Being Solved

Without abstraction, each route would: - Create its own HTTP client - Duplicate timeout logic - Handle errors inconsistently

This leads to brittle, unmaintainable code.

3.2 Base HTTP Client (`BaseServiceClient`)

A reusable async HTTP client was introduced using `httpx.AsyncClient`.

Responsibilities: - Enforce timeouts - Centralize request logic - Provide a single change point for retries/logging later

Why this is the best approach: - Implements the **Anti-Corruption Layer** pattern - Protects the gateway from downstream instability - Allows swapping HTTP libraries without touching routes

3.3 Service-Specific Clients

Each downstream service has its own client:

- AuthServiceClient
- UserServiceClient
- JobServiceClient

These clients: - Encapsulate service-specific paths - Expose semantic methods (e.g., `get_jobs()`)

Why this matters: - Routes remain clean and readable - URLs and endpoints are hidden implementation details - Improves testability and maintainability

This follows **domain-oriented design**, not endpoint-oriented design.

3.4 Async + Timeouts

The gateway uses **async HTTP calls** with enforced timeouts.

Why async: - Gateways are IO-bound - Higher concurrency with fewer resources - Industry-standard for API gateways

Why timeouts are mandatory: - Prevent cascading failures - Protect gateway resources - Ensure system resilience

4. Why Routes Were Not Written Yet

Routes were intentionally postponed until the foundation was ready.

Reasoning: - Infrastructure correctness before behavior - Prevents tight coupling - Encourages clean abstractions

This reflects a **senior engineering mindset**: *design seams before implementing features*.

5. Industry References & Learning Sources

The design choices align with the following authoritative sources:

- FastAPI – Bigger Applications <https://fastapi.tiangolo.com/tutorial/bigger-applications/>
- FastAPI – Dependencies & Service Layers <https://fastapi.tiangolo.com/tutorial/dependencies/>

- HTTPX Official Documentation <https://www.python-httpx.org/>
 - Martin Fowler – Anti-Corruption Layer <https://martinfowler.com/bliki/AntiCorruptionLayer.html>
 - 12-Factor App – Configuration <https://12factor.net/config>
 - Netflix Engineering Blog (API Gateway & Resilience Patterns) <https://netflixtechblog.com/>
-

6. Interview-Ready Summary Statement

“I designed the API Gateway as a thin orchestration layer, using a centralized async HTTP client abstraction with enforced timeouts and service-specific clients. This keeps routing logic decoupled from transport concerns and allows consistent resilience handling across services.”

This statement accurately reflects the architecture implemented in **Week 2 – Day 1 & Day 2**.

7. Status

- ☒ Week 2 – Day 1: Complete
- ☒ Week 2 – Day 2: Complete

The system is now ready for **Day 3 – First Real Routing (Job Service)**.